

Microservicio de Reseñas

Cómo funciona y cómo lo consume el sitio Django

Proyecto Biblioteca Comunitaria
Trabajo práctico de Desarrollo Web

Componente	Tecnología	Dónde vive
Sitio web	Django 6.1.1	sitio/
Microservicio	FastAPI 0.141.1	microservicio_resenas/
Base del sitio	PostgreSQL (Supabase)	La nube
Base de reseñas	PostgreSQL (Supabase)	La nube
Despliegue	Vercel Services	Un dominio, un deploy

1. Qué es este microservicio y por qué existe

Un **microservicio** es una aplicación pequeña e independiente que resuelve una sola responsabilidad y que se comunica con el resto del sistema por la red, normalmente por HTTP. No comparte código ni base de datos con quien lo consume: solo expone un contrato público, que son sus URLs y el formato de sus respuestas.

En este proyecto el microservicio tiene una única responsabilidad: **guardar y devolver las reseñas de los libros**. Nada más. No sabe qué es un préstamo, no sabe qué es un lector, ni siquiera sabe qué es un libro: para él un libro es apenas un número, el `libro_id`.

El requisito que cumple

La consigna pedía que una vista consumiera un microservicio propio, desplegado en la nube, y que ese microservicio accediera a una base de datos distinta de SQLite. Las reseñas son justamente el dato que **no** vive en la base local: viven en Supabase y solo se alcanzan a través de la API.

La idea central

Las reseñas NO son un modelo de Django. No existe una clase `Resena(models.Model)` en ninguna parte del proyecto, y eso es deliberado. Si existiera, Django estaría leyendo directamente la base del microservicio y ya no habría dos servicios: habría uno solo con dos conexiones.

Por qué no hay un modelo de Django para las reseñas

Es la pregunta más común y vale la pena responderla con precisión. El modelo de las reseñas **sí existe**, pero está definido dentro del microservicio, en dos lugares:

Qué define	Dónde está	Qué establece
La tabla	<code>microservicio_resenas/schema.sql</code>	Las columnas y sus tipos en PostgreSQL
El contrato de entrada	<code>ResenaNueva</code> , en <code>main.py</code>	Qué acepta el POST y con qué validaciones

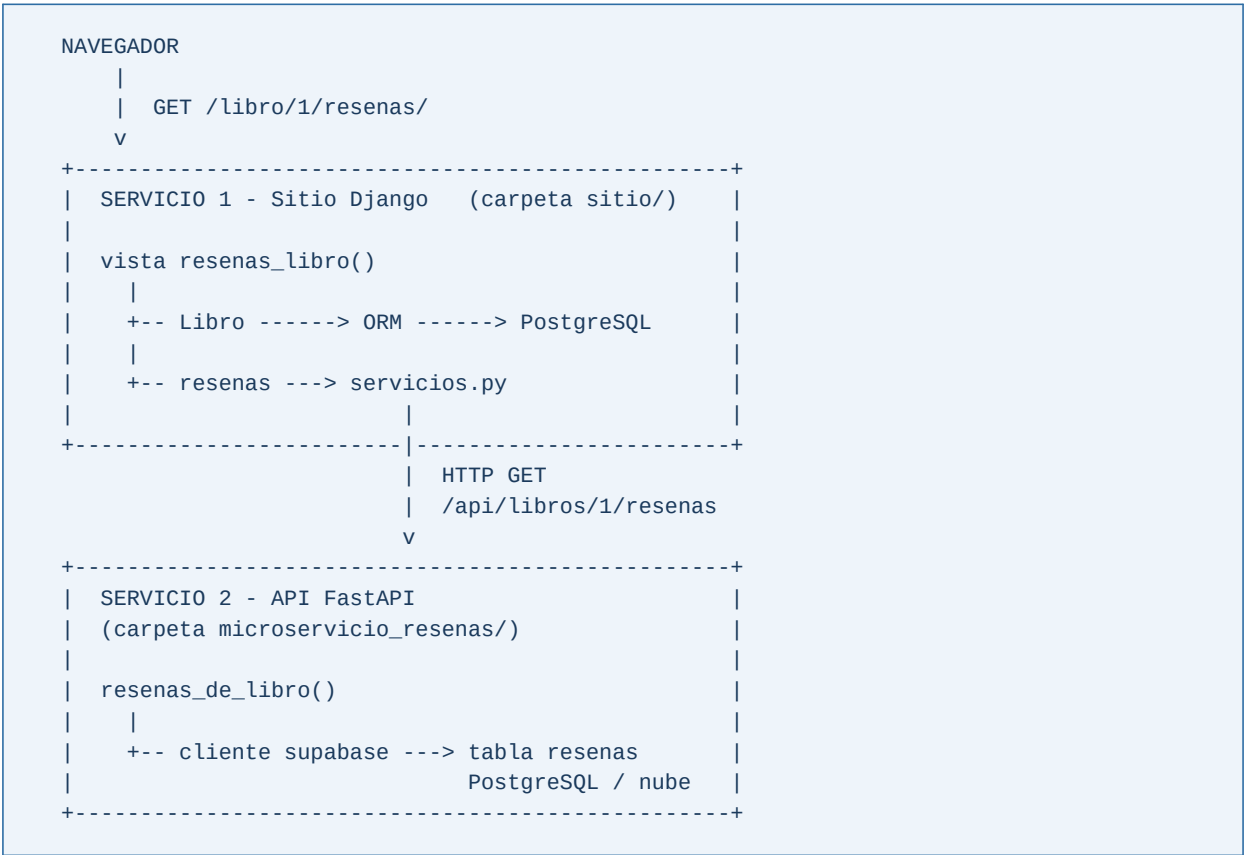
Si Django tuviera su propio modelo apuntando a esa misma tabla, habría **dos definiciones del mismo esquema en dos lugares distintos**. El día que el microservicio agregue una columna, las migraciones de Django no se enterarían y algo se rompería sin aviso. Por eso la regla es dura:

Regla de aislamiento

La base de datos de un servicio es privada. Solo se toca a través de su API. Django ni siquiera tiene las credenciales de esa base: lo único que conoce es una URL.

2. La arquitectura completa

Este es el recorrido de una petición desde que alguien abre el navegador hasta que ve las reseñas en pantalla:



Los dos servicios comparten dominio y se despliegan juntos, pero **se construyen por separado**, cada uno con sus propias dependencias. Compartir dominio no los convierte en una sola aplicación: el dominio es la puerta de entrada, no el programa.

Dato	Dónde vive	Cómo lo obtiene Django
Libros, autores, categorías	PostgreSQL del sitio	ORM de Django
Lectores y préstamos	PostgreSQL del sitio	ORM de Django
Reseñas y puntajes	Tabla resenas (Supabase)	HTTP contra el microservicio

Cómo se reparte el tráfico

El archivo `vercel.json` declara los dos servicios y decide qué URL atiende cada uno:

`vercel.json`

```
{
  "services": {
    "sitio": { "root": "sitio/", "entrypoint": "biblioteca.wsgi:application" },
    "api": { "root": "microservicio_resenas/", "entrypoint": "main:app" }
  },
  "rewrites": [
    { "source": "/api/(.*)", "destination": { "service": "api" } },
    { "source": "/(.*)", "destination": { "service": "sitio" } }
  ]
}
```

URL	La atiende	Devuelve
/	sitio	El catálogo, en HTML
/libro/1/resenas/	sitio	HTML con las reseñas pedidas a la API
/api/libros/1/resenas	api	JSON crudo desde Supabase
/api/salud	api	{"estado": "ok"}

3. El microservicio, parte por parte

Todo el microservicio es un único archivo: `microservicio_resenas/main.py`. Vamos por bloques.

3.1 Las credenciales

```
def variable_obligatoria(nombre):
    """Lee una variable de entorno obligatoria y falla con un mensaje legible."""
    valor = os.environ.get(nombre)
    if not valor:
        raise RuntimeError(f"Falta la variable de entorno {nombre}. ...")
    return valor

SUPABASE_URL = variable_obligatoria("SUPABASE_URL")
SUPABASE_SERVICE_KEY = variable_obligatoria("SUPABASE_SERVICE_KEY")

supabase: Client = create_client(SUPABASE_URL, SUPABASE_SERVICE_KEY)
```

Las claves **nunca se escriben en el código**: llegan por variables de entorno, que en el servidor se cargan desde el panel de la plataforma y en local desde el archivo `.env`, que no se versiona.

Esto ocurre al **importar** el módulo, no dentro de una función. Si falta una credencial, el servicio no arranca. Es intencional: es preferible que falle al iniciar, de forma ruidosa, antes que arranque a medias y falle en el primer pedido de un usuario.

Cómo se ve ese fallo en producción

Si el módulo explota al importarse, **todas** las rutas devuelven error, incluso una tan simple como `/api/salud`, que no toca ni la base ni la red. Ese síntoma —error en absolutamente todo— es la pista de que el problema no está en ninguna ruta, sino en el arranque del módulo.

3.2 El prefijo /api

```
app = FastAPI(title="Microservicio de Reseñas", version="1.0.0")

api = APIRouter(prefix="/api")
...
app.include_router(api)
```

Todas las rutas cuelgan de `/api` porque el servicio comparte dominio con el sitio. Cuando Vercel enruta una petición a un servicio, **le entrega la ruta completa**: un pedido a `/api/salud` le llega a FastAPI como `/api/salud`, no como `/salud`. Si el prefijo no estuviera declarado acá, todas las rutas darían 404.

3.3 El contrato de entrada

```
class ResenaNueva(BaseModel):
    """Datos que el cliente envía para crear una reseña."""

    libro_id: int = Field(..., ge=1)
    lector: str = Field(..., min_length=1, max_length=100)
    puntaje: int = Field(..., ge=1, le=5)
    comentario: str = Field("", max_length=1000)
```

Esto es un modelo de **Pydantic**, y cumple el papel que en Django cumpliría un formulario. FastAPI lo usa para validar automáticamente: si alguien manda un puntaje de 9, la petición se rechaza con un error 422 y el código del endpoint **nunca llega a ejecutarse**. No hay que escribir un solo `if` de validación.

3.4 El endpoint que consume Django

De los cuatro endpoints, este es el que usa la vista:

`microservicio_resenas/main.py`

```
@api.get("/libros/{libro_id}/resenas")
def resenas_de_libro(libro_id: int):
    """Devuelve las reseñas de un libro con su promedio de puntaje."""
    try:
        respuesta = (
            supabase.table(TABLA)
            .select("*")
            .eq("libro_id", libro_id)
            .order("creada_en", desc=True)
            .execute()
        )
    except Exception as error:
        raise HTTPException(status_code=502, detail=f"Error de Supabase: {error}")

    resenas = respuesta.data
    promedio = None
    if resenas:
        promedio = round(sum(r["puntaje"] for r in resenas) / len(resenas), 2)

    return {
        "libro_id": libro_id,
        "cantidad": len(resenas),
        "promedio": promedio,
        "resenas": resenas,
    }
```

Tres cosas para notar:

1. El `libro_id: int` de la firma no es decorativo: FastAPI convierte el texto de la URL a entero y, si no puede, responde 422 solo.
2. El **promedio se calcula acá**, no en Django ni en el template. El servicio devuelve el dato ya listo para mostrar.
3. Un libro sin reseñas **no es un error**: devuelve `cantidad: 0` y `promedio: null`, con un 200. Una lista vacía es un resultado válido.

Los cuatro endpoints

Método y ruta	Qué hace
GET /api/	Se describe a sí mismo y lista sus endpoints
GET /api/salud	Health check: responde si el servicio está vivo
GET /api/resenas	Todas las reseñas, de la más nueva a la más vieja
GET /api/libros/{id}/resenas	Reseñas de un libro y su promedio
POST /api/resenas	Crea una reseña nueva

FastAPI genera además una documentación interactiva sola, en `/api/docs`, donde se pueden probar los endpoints desde el navegador.

3.5 La tabla en Supabase

`microservicio_resenas/schema.sql`

```
create table if not exists public.resenas (  
    id          bigint generated always as identity primary key,  
    libro_id    integer      not null,  
    lector      text         not null,  
    puntaje     integer      not null check (puntaje between 1 and 5),  
    comentario  text         not null default '',  
    creada_en   timestampz   not null default now()  
);  
  
create index if not exists resenas_libro_id_idx on public.resenas (libro_id);  
  
alter table public.resenas enable row level security;
```

El check (puntaje between 1 and 5) repite a propósito la validación que ya hace Pydantic. Son dos defensas en capas distintas: Pydantic protege de un cliente mal programado, el check protege de cualquiera que escriba en la tabla sin pasar por la API.

Seguridad de la tabla

Row Level Security queda activado y **sin políticas públicas**: con la clave anónima nadie puede leer ni escribir. Solo el microservicio atraviesa esa barrera, porque usa la clave `service_role`, que vive como variable de entorno del servidor y nunca sale de ahí.

4. El lado de Django: cómo lo consume

Del lado del sitio hay tres capas bien separadas, y cada una sabe una cosa distinta:

Capa	Archivo	Qué sabe
Cliente HTTP	libros/servicios.py	Que las reseñas son remotas y cómo pedirlas
Vista	libros/views.py	Que hay que combinar datos locales y remotos
Template	libros/templates/libros/resenas.html	Nada del origen: solo muestra el context

4.1 El cliente HTTP

servicios.py es el **único módulo de todo el proyecto que sabe que las reseñas viven fuera**. Si mañana el microservicio cambiara de dirección o de formato, este es el único archivo a tocar.

[sitio/libros/servicios.py](#)

```
class MicroservicioNoDisponible(Exception):
    """El microservicio no respondió, tardó demasiado o devolvió algo inesperado."""

def _pedir(url, datos=None):
    """Hace la petición HTTP y devuelve el JSON ya parseado."""
    cuerpo = None
    cabeceras = {"Accept": "application/json"}

    if datos is not None:
        cuerpo = json.dumps(datos).encode("utf-8")
        cabeceras["Content-Type"] = "application/json"

    petition = urllib.request.Request(url, data=cuerpo, headers=cabeceras)

    try:
        with urllib.request.urlopen(
            petition, timeout=settings.MICROSERVICIO_RESENAS_TIMEOUT
        ) as respuesta:
            return json.loads(respuesta.read().decode("utf-8"))
    except (urllib.error.URLError, TimeoutError, ValueError) as error:
        raise MicroservicioNoDisponible(str(error)) from error

def obtener_resenas(libro_id):
    url = f"{settings.MICROSERVICIO_RESENAS_URL}/libros/{libro_id}/resenas"
    return _pedir(url)
```

Usa `urllib`, de la biblioteca estándar, en lugar de `requests`: así el sitio no suma una dependencia solo para hacer un pedido HTTP.

Lo importante es la **traducción de errores**. Una caída de red, un timeout y una respuesta que no es JSON son tres fallas técnicas muy distintas, pero para la vista significan exactamente lo mismo: *no tengo las reseñas*. Por eso las tres se convierten en una sola excepción propia. La vista no necesita saber nada de `urllib`.

4.2 La vista

Sigue el mismo patrón de tres pasos que todas las vistas del proyecto. La diferencia es que consume **dos fuentes**: el ORM para el libro y el microservicio para las reseñas.

`sitio/libros/views.py`

```
def resenas_libro(request, libro_id):
    """Combina un Libro del ORM con sus reseñas traídas del microservicio."""
    # 1. consumir el modelo y, por HTTP, el microservicio
    libro = get_object_or_404(Libro, pk=libro_id)
    mensaje = ""

    if request.method == "POST":
        ...
        servicios.crear_resena(libro_id=libro.id, lector=lector, ...)

    try:
        datos = servicios.obtener_resenas(libro.id)
        error = ""
    except servicios.MicroservicioNoDisponible as fallo:
        datos = {}
        error = f"El microservicio de reseñas no está disponible ({fallo})"

    # 2. armar el context como variable con nombre
    context = {
        "libro": libro,
        "resenas": datos.get("resenas", []),
        "cantidad": datos.get("cantidad", 0),
        "promedio": datos.get("promedio"),
        "url_microservicio": settings.MICROSERVICIO_RESENAS_URL,
        "mensaje": mensaje,
        "error": error,
    }

    # 3. enviarlo al template
    return render(request, "libros/resenas.html", context)
```

Por qué el try/except no es opcional

Una consulta al ORM local falla, casi siempre, solo si hay un error de programación. Una llamada HTTP a otra máquina falla por motivos que **no se controlan**: se cayó la red, la plataforma reinició el servicio, la base está lenta, alguien cambió la URL.

Degradación controlada

Toda llamada remota se asume falible. Si el microservicio no responde, la página se sigue mostrando con un aviso y los datos del libro intactos; no devuelve un error 500. Esa es la diferencia práctica entre leer de una base local y leer de un servicio externo, y es el motivo de que el proyecto tenga un test dedicado a verificar justamente este caso.

Por lo mismo existe un **timeout** de 5 segundos. Sin él, la vista quedaría colgada esperando a un servidor que quizá no conteste nunca, y con ella el proceso que atiende a ese usuario.

4.3 El template

El template no sabe —ni tiene por qué saber— de dónde salieron los datos. Recibe una lista en el context y la recorre, igual que cualquier otra vista del proyecto. Esa ignorancia es justamente la señal de que la separación está bien hecha.

[sitio/libros/templates/libros/resenas.html](#)

```
{% if resenas %}
    <table border="1">
        <tr><th>Lector</th><th>Puntaje</th><th>Comentario</th><th>Fecha</th></tr>
        {% for resena in resenas %}
            <tr>
                <td>{{ resena.lector }}</td>
                <td>{{ resena.puntaje }}</td>
                <td>{{ resena.comentario }}</td>
                <td>{{ resena.creada_en }}</td>
            </tr>
        {% endfor %}
    </table>
{% else %}
    <p>Este libro todavía no tiene reseñas.</p>
{% endif %}
```

Un detalle: `resena.lector` no es un atributo de un objeto de Python, es una clave de un diccionario que vino de un JSON. La sintaxis del template es idéntica en los dos casos, y por eso el cambio de origen resulta invisible desde acá.

5. El recorrido completo de una petición

Poniendo todo junto, esto es lo que pasa cuando alguien abre `/libro/1/resenas/`:

#	Qué ocurre
1	El navegador pide <code>/libro/1/resenas/</code> .
2	Vercel mira sus reglas: la ruta no empieza con <code>/api</code> , así que va al servicio sitio .
3	Django resuelve la URL y llama a <code>resenas_libro(request, 1)</code> .
4	La vista busca el libro con el ORM: <code>get_object_or_404(Libro, pk=1)</code> . Si no existe, 404 y se terminó.
5	La vista llama a <code>servicios.obtener_resenas(1)</code> .
6	El cliente arma la URL y hace GET <code>.../api/libros/1/resenas</code> , con 5 segundos de plazo.
7	Vercel ve el prefijo <code>/api</code> y manda la petición al servicio api .
8	FastAPI convierte el 1 de la URL a entero y ejecuta <code>resenas_de_libro(1)</code> .
9	El endpoint consulta Supabase, calcula el promedio y devuelve el JSON.
10	El cliente parsea ese JSON y devuelve un diccionario. Si algo falló, lanza <code>MicroservicioNoDisponible</code> .
11	La vista arma el context con el libro y las reseñas.
12	El template lo renderiza y el navegador recibe el HTML.

Qué pasa si el microservicio está caído

El paso 6 falla, el 10 lanza la excepción, y la vista la atrapa. El context se arma igual, con una lista vacía de reseñas y un mensaje de error. **El usuario ve la página, con los datos del libro y un aviso donde irían las reseñas.** No ve una pantalla de error.

Cómo sabe Django a qué dirección llamar

`sitio/biblioteca/settings.py`

```
def _url_del_microservicio():
    explicita = os.environ.get('MICROSERVICIO_RESENAS_URL')
    if explicita:
        return explicita
    dominio = (
        os.environ.get('VERCEL_PROJECT_PRODUCTION_URL')
        or os.environ.get('VERCEL_URL')
    )
    if dominio:
        return f'https://{dominio}/api'
    return 'http://127.0.0.1:8001/api'

MICROSERVICIO_RESENAS_URL = _url_del_microservicio().rstrip('/')
MICROSERVICIO_RESENAS_TIMEOUT = 5 # segundos
```

En desarrollo apunta al puerto 8001 de la propia máquina. En el servidor deduce el dominio público del proyecto y le agrega `/api`, así que no hay nada que configurar a mano. Y siempre se puede forzar otra dirección con una variable de entorno, sin tocar una línea de código.

6. Probarlo y entenderlo

Ver los dos lados de la misma información

La forma más clara de comprobar que son dos sistemas distintos es pedir el mismo dato por las dos vías:

Abrí esta URL	Y vas a ver
/libro/1/resenas/	La página del sitio, con las reseñas en una tabla HTML
/api/libros/1/resenas	El JSON crudo que la vista consumió para armarla
/api/docs	La documentación interactiva que FastAPI genera sola

Son los mismos datos, servidos por dos programas distintos, en dos formatos distintos, desde el mismo dominio.

Levantarlo en local

```
# Terminal 1 – el microservicio
cd microservicio_resenas
set -a; . ../.env; set +a
venv/bin/uvicorn main:app --reload --port 8001

# Terminal 2 – el sitio
cd sitio
venv/bin/python manage.py runserver
```

Hacen falta **dos terminales**, y eso también enseña algo: son dos procesos independientes. Se puede apagar el microservicio con el sitio andando y ver la degradación controlada en vivo.

Lo que conviene llevarse

1. El dato define el límite

Las reseñas viven fuera de la base del sitio. Esa decisión es la que crea el microservicio: todo lo demás —la API, el cliente HTTP, el manejo de errores— es consecuencia de ella.

2. Lo remoto falla, y hay que preverlo

Timeout y try/except en toda llamada de red. Un servicio caído degrada la página; no la rompe.

3. Un solo lugar sabe del exterior

servicios.py concentra todo lo que sabe que las reseñas son remotas. La vista y el template quedan igual de simples que cualquier otro del proyecto.

4. Las credenciales no viajan

Django no tiene las claves de la base de las reseñas. Solo conoce una URL. Ese aislamiento es el beneficio concreto que se compra al separar los servicios.

Archivos de referencia: `microservicio_resenas/main.py`, `microservicio_resenas/schema.sql`,
`sitio/libros/servicios.py`, `sitio/libros/views.py`, `vercel.json`

7. Glosario de términos

Los términos que aparecen en este documento, agrupados por tema. Los ejemplos salen del propio proyecto.

Arquitectura

Microservicio	Aplicación pequeña e independiente que resuelve una sola responsabilidad y se comunica con el resto por la red. No comparte código ni base de datos con quien la consume. Acá: la API de reseñas.
Monolito	Lo opuesto: una única aplicación que hace todo y comparte una sola base de datos. El sitio Django, por sí solo, es un monolito, y para este tamaño está bien.
Acoplamiento	Cuánto depende una parte del sistema de los detalles internos de otra. Que Django conozca solo una URL, y no el esquema de la base de reseñas, es bajo acoplamiento.
Contrato	Lo que un servicio promete públicamente: sus URLs y el formato de sus respuestas. Mientras el contrato no cambie, el microservicio puede reescribirse por dentro sin que Django se entere.
Degradación controlada	Que el sistema siga funcionando, con menos información, cuando una parte falla. Si la API se cae, la página del libro se muestra igual con un aviso en lugar de las reseñas.
Shared database antipattern	El error de que dos servicios escriban la misma tabla con definiciones propias del esquema. Tarde o temprano se desincronizan. Es la razón de que no exista un modelo Resena en Django.

Web y HTTP

HTTP	El protocolo con el que se piden y devuelven cosas en la web. Es el canal por el que la vista de Django habla con el microservicio.
GET / POST	Los dos métodos HTTP que usa el proyecto. GET pide datos sin modificar nada; POST envía datos para crear algo.
JSON	Formato de texto para intercambiar datos, con llaves y listas. Es lo que devuelve el microservicio y lo que la vista convierte en un diccionario de Python.
API	Interfaz de programación: el conjunto de URLs que un servicio expone para que otros programas lo usen. Devuelve datos, no páginas.
Endpoint	Una URL concreta de una API, con su método. Por ejemplo GET /api/libros/1/resenas.
Timeout	El tiempo máximo que se espera una respuesta antes de darla por perdida. Acá son 5 segundos. Sin timeout, una vista puede quedar colgada para siempre.
Health check	Endpoint trivial que solo responde si el servicio está vivo. Acá es /api/salud. Lo usan las plataformas para monitorear.

Códigos de estado que usa el proyecto

200 OK	Todo bien. Un libro sin reseñas también devuelve 200: una lista vacía es un resultado válido, no un error.
201 Created	Se creó algo. Lo devuelve POST /api/resenas.
404 Not Found	No existe lo pedido. Lo produce get_object_or_404 cuando el libro no está.
422 Unprocessable	Los datos enviados no pasaron la validación. FastAPI lo devuelve solo, por ejemplo con un puntaje de 9.
500 Internal Error	El servidor se rompió. Es lo que el try/except de la vista evita cuando la API no responde.
502 Bad Gateway	Un servicio intermedio falló. El microservicio lo devuelve si Supabase no contesta.

Django

Vista	Función que recibe una petición, consigue los datos y devuelve una respuesta. En este proyecto siempre sigue tres pasos: consumir datos, armar el context, renderizar el template.
Template	Plantilla HTML con huecos que se rellenan con los datos del context. No consulta la base de datos.
Context	Diccionario que la vista le pasa al template. Es la frontera entre la lógica y la presentación.
ORM	<i>Object-Relational Mapper</i> . Traduce clases de Python a tablas SQL, para no escribir consultas a mano. Libro.objects.filter(...) es el ORM trabajando.
Modelo	Clase de Python que representa una tabla. Libro, Autor y Prestamo son modelos; las reseñas no .
Migración	Archivo que describe un cambio en la estructura de la base, para poder aplicarlo de forma reproducible. Las genera makemigrations.
Excepción propia	Una clase de error definida por el proyecto, como MicroservicioNoDisponible. Permite que la vista atrape el problema sin saber nada de urllib.
urllib	Librería de la biblioteca estándar de Python para hacer pedidos HTTP. Se usa en lugar de requests para no sumar dependencias.
WSGI	El estándar que conecta un servidor web con una aplicación Python sincrónica. biblioteca/wsgi.py expone el objeto application que la plataforma ejecuta.

FastAPI y el microservicio

FastAPI	Framework de Python para construir APIs. Valida los datos y genera la documentación automáticamente a partir de los tipos que se declaran.
ASGI	El equivalente asincrónico de WSGI. FastAPI es una aplicación ASGI; el objeto se llama <code>app</code> .
Pydantic	Librería de validación por tipos. <code>ResenaNueva</code> es un modelo de Pydantic: cumple el papel de un formulario de Django.
APIRouter	Agrupador de rutas de FastAPI. Con <code>APIRouter(prefix="/api")</code> todas las rutas del servicio cuelgan de <code>/api</code> sin repetirlo una por una.
uvicorn	El servidor que ejecuta la aplicación ASGI. En local: <code>uvicorn main:app --port 8001</code> .
Swagger / /docs	La documentación interactiva que FastAPI genera sola, donde se pueden probar los endpoints desde el navegador.

Bases de datos

SQLite	Base de datos que vive en un único archivo. Perfecta para desarrollo; inservible en un servidor sin disco persistente, que es el motivo de haber migrado a PostgreSQL.
PostgreSQL	Motor de base de datos que corre como servicio aparte, en su propio servidor. La usan el sitio y el microservicio.
Supabase	Servicio que ofrece PostgreSQL administrado en la nube, más una API HTTP para consultarlo. El microservicio usa esa API; Django se conecta al Postgres directo.
Connection pooler	Intermediario que reutiliza conexiones a la base en lugar de abrir una nueva cada vez. En Supabase hay que usar el Session pooler : la conexión directa es IPv6 y la plataforma no la alcanza.
RLS	<i>Row Level Security</i> . Mecanismo de PostgreSQL que decide, fila por fila, quién puede leer o escribir. Acá está activado sin políticas públicas : con la clave anónima nadie entra.
service_role / anon	Las dos claves de Supabase. La anon es pública y respeta RLS; la service_role lo atraviesa y es secreta: vive solo en el servidor.
Idempotente	Que se puede ejecutar varias veces con el mismo resultado. El <code>seed.py</code> lo es: correrlo dos veces no duplica datos.

Despliegue

Serverless	Modelo donde el código se ejecuta en respuesta a una petición, sin un servidor encendido de forma permanente. No hay disco que persista entre pedidos: de ahí que SQLite no sirva.
Función	La unidad que se ejecuta en serverless. Toda la aplicación Django se empaqueta en una sola función.
Cold start	La demora extra del primer pedido, cuando la función tiene que inicializarse desde cero porque no había ninguna instancia despierta.
Build	El proceso que instala dependencias y arma el paquete que se va a ejecutar. Ocurre una vez por despliegue, no en cada pedido.
Deployment	Una versión concreta y inmutable del proyecto ya construida. No se modifica: cada cambio genera uno nuevo.
Alias / dominio	El nombre público que apunta a un deployment. Un deployment nuevo no se ve hasta que el alias lo apunta: si la URL no cambia pese a los cambios, este es el primer lugar donde mirar.
Promote	La acción de hacer que los alias apunten a un deployment ya construido.
Entrypoint	El archivo y la variable que la plataforma carga para arrancar la aplicación: <code>libreria.wsgi:application</code> para el sitio y <code>main:app</code> para la API.
Rewrite	Regla que decide, según la ruta pedida, qué servicio la atiende. Las de este proyecto mandan <code>/api/*</code> a la API y el resto al sitio.
Root Directory	La carpeta que la plataforma toma como raíz del proyecto. Con Services va vacía, porque el reparto lo define <code>vercel.json</code> .

Variable de entorno	Valor que se le pasa al programa desde fuera del código, como una credencial o una URL. Permite cambiar la configuración sin tocar ni desplegar código nuevo.
collectstatic	Comando de Django que junta los archivos estáticos en una carpeta para que el servidor los sirva. La plataforma lo corre sola durante el build.